

Docking control (Docking Control)

PHỤ THUỘC A/05-lan-truyen-sai-so-pixel-sang-pose (mục 8, phân loại nguồn ngẫu nhiên và hệ thống — toàn bộ lập luận teach-by-showing dựa vào đó); A/06-hieu-ung-vat-ly-va-quang-hoc (mục 7, bốn hiệu ứng biến mất khi vận tốc bằng không — cơ sở vật lý của stop-and-go); B/09-multi-marker-constellation (constellation là thứ mà bộ điều khiển nhìn vào).

TÓM TẮT

Chương này chứa mẹo lớn nhất của cả cây, và mẹo ấy không phải một thuật toán mà là một cách đặt lại bài toán. Quan sát then chốt: robot cập dock *không cần biết nó ở đâu trong hệ tọa độ tuyệt đối*; nó cần *trở về đúng chỗ nó đã từng ở*. Hai yêu cầu ấy khác nhau về bản chất — một là độ chính xác (accuracy), một là độ lặp lại (repeatability) — và yêu cầu thứ hai dễ hơn một bậc, vì mọi sai số *hệ thống* xuất hiện ở cả pose hiện tại lẫn pose mục tiêu và triệt tiêu lẫn nhau. Đó là teach-by-showing, nội dung của mục 3. Bốn công cụ còn lại: IBVS điều khiển thẳng trên tọa độ ảnh để tránh hẳn bài toán pose; stop-and-go xoá các hiệu ứng phụ thuộc vận tốc; coarse-to-fine chia hành trình theo độ chính xác cần thiết; và côn dẫn hướng cơ khí — thứ mà một cây tri thức về thị giác phải trung thực thừa nhận là thường rẻ nhất. Nếu chỉ nhớ một điều: *đừng giải bài toán accuracy nếu bài toán thật là repeatability*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **độ chính xác (accuracy)** — mức độ gần của giá trị đo tới giá trị thật. Với robot, đó là khả năng tới được một tọa độ được chỉ định trong một hệ quy chiếu tuyệt đối [1].
- **độ lặp lại (repeatability)** — mức độ gần giữa các lần thực hiện cùng một lệnh. Robot công nghiệp thường có độ lặp lại tốt hơn độ chính xác cả bậc độ lớn, và tiêu chuẩn phân biệt rõ hai đại lượng [1].
- **teach-by-showing** — ghi lại trạng thái cảm biến tại vị trí đích một lần (bằng cách đưa robot tới đó bằng tay hoặc bằng đồ gá), rồi về sau điều khiển để tái tạo đúng trạng thái ấy, thay vì điều khiển tới một tọa độ tính toán.
- **IBVS** — *image-based visual servoing*: điều khiển trực tiếp trên các đặc trưng ảnh (tọa độ điểm trên ảnh), với sai lệch định nghĩa trong không gian ảnh chứ không trong không gian pose [2].
- **PBVS** — *position-based visual servoing*: ước lượng pose từ ảnh rồi điều khiển trong không gian pose. Trực quan hơn nhưng nhạy hơn với sai số hiệu chuẩn vì pose là một đại lượng *suy ra*, không phải đại lượng đo.
- **ma trận tương tác (interaction matrix)** — ma trận L nối vận tốc của camera với tốc độ thay đổi của đặc trưng ảnh: $\dot{s} = L v$. Nó là Jacobian của bài toán điều khiển, và cấu trúc của nó quyết định hành vi của IBVS [2, §II].
- **nonholonomic** — ràng buộc trên *vận tốc* mà không rút gọn thành ràng buộc trên vị trí. Robot vi sai hai bánh là ví dụ: nó không đi ngang được tại chỗ, dù nó tới được mọi vị trí.
- **chamfer / côn dẫn hướng** — mặt vát ở miệng lỗ hoặc đầu chốt, biến một sai lệch vị trí thành một lực dẫn hướng thay vì một va chạm. Lý thuyết của nó thuộc ngành lắp ráp tự động [3].

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn có sai số hệ thống 3 mm do intrinsic chưa hoàn hảo và hand-eye chưa chuẩn. Có một cách làm cho 3 mm ấy gần như biến mất mà không cần hiệu chuẩn lại gì cả. Cách ấy là gì, và nó “gần như” chứ không phải “hoàn toàn” vì sao?
- Q2. IBVS điều khiển trên tọa độ điểm ảnh và không bao giờ tính pose. Nếu hai vấn đề lớn của cả cây mà nó tránh được hoàn toàn nhờ việc không tính pose, và nêu cái giá phải trả.
- Q3. Robot vi sai không đi ngang được. Nếu nó đang ở đúng trước dock nhưng lệch sang phải 2 cm, vì sao việc sửa 2 cm ấy khó hơn nhiều so với sửa 2 cm theo chiều tiến lùi — và định lý nào nói rằng không có luật điều khiển trơn nào giải quyết được?
- Q4. Dừng hẳn robot lại 0,3 giây trước khi đo có hai lợi ích hoàn toàn khác nhau về bản chất. Nêu cả hai, và nói cái nào lớn hơn.
- Q5. Một kỹ sư cơ khí đề xuất thêm một cái côn dẫn hướng vào miệng dock, và nói rằng như vậy thì góc chỉ cần chính xác 15 mm thay vì 5 mm. Vì sao đề xuất ấy thường đúng, và trong trường hợp nào thì nó không giúp gì?

1.1 Đặt vấn đề

Mọi chương trước trong cây đều tìm cách làm cho một con số chính xác hơn. Chương này hỏi một câu khác: *con số ấy có thật sự cần chính xác không?*

Bài toán như thường được phát biểu: robot phải cập vào dock với sai lệch dưới 5 mm và $0,5^\circ$. Phát biểu ấy nghe như một yêu cầu về *độ chính xác* — robot phải biết nó đang ở đâu so với dock, chính xác tới 5 mm. Và nếu đó là yêu cầu thật, thì ngân sách sai số của C/15 phải tính mọi mất xích: intrinsic, méo, constellation, hand-eye, và mọi thứ khác — vì mỗi mất xích đóng góp vào việc “biết mình ở đâu”.

Nhưng hãy nhìn kỹ hơn vào việc robot thật sự cần làm. Nó cần đưa đầu sọc của nó chạm đúng vào tiếp điểm trên dock. Nó *không* cần biết tọa độ của mình trong bất kỳ hệ quy chiếu nào. Nếu ta từng đưa robot tới đúng vị trí cập chuẩn — bằng tay, bằng đồ gá — và ghi lại camera nhìn thấy gì tại đó, thì nhiệm vụ trở thành: *đưa camera về trạng thái nhìn thấy đúng như vậy*.

Sự khác biệt giữa hai phát biểu ấy không phải chuyện chữ nghĩa. Tiêu chuẩn robot công nghiệp phân biệt rạch ròi *độ chính xác* và *độ lặp lại* [1], và với hầu hết robot thì độ lặp lại tốt hơn độ chính xác cả bậc độ lớn. Lý do là thế này: độ chính xác đòi mô hình đúng — mọi tham số hình học phải khớp với thực tế. Độ lặp lại chỉ đòi mô hình *nhất quán* — nó có thể sai, miễn là sai giống nhau ở cả hai lần.

Và đây là chỗ nối với A/05 mục 8. Các nguồn sai số chia làm hai loại: ngẫu nhiên (đổi giữa các lần đo) và hệ thống (không đổi). Trong bài toán độ chính xác, cả hai đều tính. Trong bài toán độ lặp lại, *chỉ nguồn ngẫu nhiên tính*, vì nguồn hệ thống xuất hiện ở cả lần dạy lẫn lần chạy và trừ đi nhau. Với một hệ thống điển hình, các nguồn hệ thống chiếm phần lớn ngân sách, nên việc loại chúng đi là một cải thiện bậc độ lớn chứ không phải một cải thiện tăng dần.

Cách hiển nhiên — và cách gần như mọi dự án đi — là giải bài toán độ chính xác: hiệu chuẩn thật kỹ, đo constellation thật kỹ, tính pose thật chính xác, rồi điều khiển tới tọa độ đích. Cách này *hoạt động* nếu bạn làm đủ kỹ, và toàn bộ Phần A cùng các chương khác của Phần B là hướng dẫn để làm đủ kỹ. Nhưng nó tốn nhiều công hơn cần thiết cho một bài toán mà bản chất là lặp lại. Chương này lập luận rằng nếu bài toán của bạn đúng là “về đúng một chỗ”, thì có một con đường ngắn hơn nhiều.

Cần nói ngay một giới hạn để không bán quá lời: teach-by-showing *không* miễn cho bạn khỏi mọi thứ. Nó triệt tiêu sai số hệ thống ở *bậc nhất* và chỉ trong lân cận của tư thế đã dạy; càng xa tư thế

ấy thì triết tiêu càng kém. Mục 3 nói rõ chỗ này, vì một mẹo được hiểu sai thì nguy hiểm hơn không biết mẹo.

TRỰC GIÁC

Hình dung việc đỗ xe vào một chỗ hẹp.

Cách thứ nhất: đo. Bạn đo chiều rộng chỗ đỗ, đo chiều dài xe, tính toạ độ mà bánh xe phải tới, rồi lái theo con số ấy. Cách này đòi mọi phép đo đúng — nếu thước của bạn sai một phân thì xe đỗ lệch một phân.

Cách thứ hai: nhớ. Bạn đã đỗ vào chỗ này một trăm lần. Bạn biết khi nào cột đèn nằm ngay góc kính chắn gió thì bánh sau đúng chỗ. Bạn không biết cái cột đèn ấy cách bánh xe bao nhiêu xăng-ti-mét, và bạn *không cần biết*. Bạn chỉ cần lái sao cho hình ảnh trước mắt trùng với hình ảnh bạn nhớ.

Cách thứ hai chính xác hơn nhiều, và lý do rất đáng nghĩ: nó không dùng phép đo nào cả, nên nó không mang sai số của phép đo nào cả. Nếu kính chắn gió của bạn hơi cong làm cột đèn trông lệch đi hai phân — thì nó cũng trông lệch đi đúng hai phân ấy vào cái lần bạn đỗ đúng, nên hai cái lệch triệt tiêu nhau.

Đó là toàn bộ ý tưởng. Sai số nào xuất hiện *giống hệt nhau* ở lần dạy và lần chạy thì tự biến mất, mà không cần ai biết nó tồn tại.

Nhưng có một giới hạn, và nó cũng thấy được bằng cùng hình ảnh. Nếu hôm nay bạn ngồi cao hơn hôm trước năm phân, thì cái cột đèn không còn nằm ở chỗ cũ trên kính nữa, và mẹo hỏng. Sai số chỉ triệt tiêu khi tình huống *đủ giống* tình huống lúc dạy. Càng lệch xa thì triết tiêu càng kém — và đó là lý do mẹo này hoạt động tốt ở đoạn cuối hành trình và kém hơn ở đoạn đầu.

1.2 Hai họ phương pháp: PBVS và IBVS

Trước khi vào mẹo chính, đặt bản đồ cho không gian lựa chọn. Có hai họ, và chúng khác nhau ở chỗ *sai lệch được định nghĩa trong không gian nào* [2].

PBVS (position-based): ước lượng pose từ ảnh, so với pose đích, được một sai lệch trong $SE(3)$, rồi điều khiển để triệt tiêu nó. Đây là cách trực quan nhất và là cách hầu hết người ta làm mặc định. Ưu điểm: sai lệch có nghĩa vật lý rõ ràng, quỹ đạo trong không gian thật dễ dự đoán và dễ ràng buộc (tránh vật cản, giữ trong vùng làm việc). Nhược điểm: nó đi qua bước ước lượng pose, nên nó thừa hưởng *mọi* vấn đề của Phần A — ambiguity, nhạy với intrinsic, nhạy với constellation.

IBVS (image-based): định nghĩa sai lệch trực tiếp trên toạ độ các điểm đặc trưng trên ảnh. Gọi $\mathbf{s}$ là vectơ chứa toạ độ ảnh của các góc marker và $\mathbf{s}^*$ là giá trị của chúng tại vị trí đích. Sai lệch là $\mathbf{e} = \mathbf{s} - \mathbf{s}^*$, và luật điều khiển cơ bản là

$$\mathbf{v} = -\lambda L^+ \mathbf{e}, \quad (1.1)$$

với L là ma trận tương tác nối vận tốc camera với tốc độ đổi của đặc trưng ảnh, L^+ là giả nghịch đảo của nó, và $\lambda > 0$ là hệ số khuếch đại [2, §II].

Đây là câu trả lời cho Q2, và hai vấn đề mà IBVS tránh hoàn toàn đáng nêu rõ.

Vấn đề thứ nhất: pose ambiguity. IBVS không bao giờ giải PnP, nên nó không bao giờ phải chọn giữa hai nghiệm. Toàn bộ A/04 và B/10 trở nên không liên quan. Đây là một lợi ích lớn và ít được nhấn mạnh đủ.

Vấn đề thứ hai: độ nhạy với hiệu chuẩn. Trong (1.1), sai số của K và của mô hình constellation chỉ vào qua ma trận L , tức chỉ ảnh hưởng tới *hướng và độ lớn của bước điều khiển*, không ảnh hưởng tới *điểm dừng*. Điểm dừng được định nghĩa bởi $\mathbf{e} = 0$, tức bởi ảnh trùng ảnh đích — một điều

kiện hoàn toàn không chứa K . Hệ quả: một L sai làm robot đi tới đích theo đường vòng vèo hơn nhưng nó *vẫn tới đúng đích*. Đây là tính chất bền vững kinh điển của IBVS, đã được nêu từ bài gốc [4].

Cái giá phải trả, và nó thật: quỹ đạo trong không gian ba chiều không được kiểm soát trực tiếp. IBVS có thể tạo ra những chuyển động kỳ lạ — trường hợp nổi tiếng nhất là “camera retreat”, khi một phép xoay lớn quanh trục quang làm camera lùi ra xa rồi tiến lại. Với robot di động trên sàn và tag ở phía trước, các bệnh lý ấy ít gặp hơn nhiều so với trường hợp cánh tay máy sáu bậc tự do, nhưng chúng vẫn nên được kiểm bằng mô phỏng. Có các phương pháp lai giải quyết chuyện này bằng cách tách phần xoay ra khỏi phần tịnh tiến [5], [6].

Bảng 1.1. PBVS, IBVS và lai: sai lệch định nghĩa ở đâu, và hệ quả.

	Sai lệch trong	Ưu	Nhược
PBVS	Không gian pose $SE(3)$	Quỹ đạo 3D dự đoán được, dễ ràng buộc	Thừa hưởng toàn bộ vấn đề của PnP: ambiguity, nhạy hiệu chuẩn
IBVS	Không gian ảnh	Không giải PnP nên không có ambiguity; điểm dừng không phụ thuộc K [4]	Quỹ đạo 3D không kiểm soát trực tiếp; có bệnh lý (camera retreat)
Lai 2,5D	Tách xoay và tịnh tiến [5]	Giữ phần lớn ưu điểm của cả hai	Phức tạp hơn; cần phân rã homography nên lại chạm vào A/θ_2

1.3 Teach-by-showing: mẹo lớn nhất

Đây là mục trung tâm của chương và nó trả lời Q1.

A. Cơ chế

Quy trình gồm hai giai đoạn.

Giai đoạn dạy, làm một lần. Đưa robot tới đúng vị trí cập chuẩn — bằng tay, bằng một đồ gá cơ khí, hoặc bằng cách để nó cập vào rồi tinh chỉnh cho tới khi tiếp điểm khớp hoàn hảo. Tại đó, ghi lại trạng thái cảm biến: hoặc là pose T^* tính được từ constellation (cho PBVS), hoặc là véctơ toạ độ ảnh s^* của mọi góc nhìn thấy (cho IBVS). Ghi lấy trung bình nhiều khung khi robot đứng yên, để s^* có nhiễu nhỏ.

Giai đoạn chạy, mỗi lần cập dock. Điều khiển để đưa trạng thái hiện tại về trùng với trạng thái đã ghi. Với PBVS, sai lệch là $T^{-1}T^*$ — một phép biến đổi tương đối. Với IBVS, sai lệch là $s - s^*$.

B. Vì sao sai số hệ thống triệt tiêu

Xét một nguồn sai số hệ thống bất kỳ — gọi nó là một hàm g biến pose thật thành pose đo được, không phụ thuộc thời gian. Nó có thể là: intrinsic sai, méo còn sót, constellation lệch CAD, hand-eye sai. Pose đo được tại vị trí đích là $\hat{T}^* = g(T_{\text{thật}}^*)$, và pose đo được tại vị trí hiện tại là $\hat{T} = g(T_{\text{thật}})$.

Bộ điều khiển triệt tiêu $\hat{T}^{-1}\hat{T}^*$. Khi nó hội tụ, $\hat{T} = \hat{T}^*$, tức $g(T_{\text{thật}}) = g(T_{\text{thật}}^*)$. Nếu g là đơn ánh — và mọi sai số nhỏ liên tục đều thế — thì $T_{\text{thật}} = T_{\text{thật}}^*$.

Đọc kết quả ấy cho kỹ, vì nó mạnh hơn vẻ ngoài: *robot tới đúng vị trí thật, bất kể g là gì*. Bạn không cần biết g , không cần đo nó, không cần mô hình hoá nó. Nó chỉ cần *không đổi* giữa lần dạy và lần chạy.

Điều kiện “không đối” là toàn bộ nội dung của phần giới hạn dưới đây, và nó là câu trả lời cho về “gần như chứ không hoàn toàn” của Q1.

C. Bốn chỗ triệt tiêu không hoàn toàn

Một, g phụ thuộc tư thế. Đây là chỗ lớn nhất. Sai số do méo còn sót phụ thuộc vị trí tag trên ảnh; bias tâm ellipse phụ thuộc góc nghiêng (A/02 mục 6); sai số do điểm chính phụ thuộc cả vị trí lẫn khoảng cách (A/01 mục 5). Khi robot ở gần tư thế đích, các phụ thuộc ấy gần như nhau và triệt tiêu tốt. Khi robot còn xa, chúng khác nhau và triệt tiêu kém. Nói bằng ngôn ngữ giải tích: triệt tiêu là ở bậc nhất quanh tư thế dạy, và phần dư là bậc hai theo khoảng cách tới tư thế ấy.

Hai, g trôi theo thời gian. Trôi nhiệt của ống kính, tag bị bắn dần, dock bị va chạm nhẹ. Mọi thứ làm g đổi giữa lần dạy và lần chạy đều phá mero. Cách phòng: dạy lại định kỳ — và đây là một lợi thế thực dụng của phương pháp, vì dạy lại rẻ hơn hiệu chuẩn lại rất nhiều.

Ba, nguồn ngẫu nhiên không triệt tiêu. Nhiều định vị góc, nhiều ảnh — những thứ đổi giữa các khung — không bị ảnh hưởng bởi mero này chút nào. Chúng vẫn ở đó và chúng vẫn phải được xử lý bằng cách lấy trung bình (B/11). Điều mero này làm là biến ngân sách từ “hệ thống cộng ngẫu nhiên” thành “chỉ ngẫu nhiên”, và vì ngẫu nhiên chia cho $\sqrt{N}$ nên phần còn lại rất dễ xử lý.

Bốn, sai số của chính lần dạy. Vị trí đích được xác định bằng chất lượng của lần dạy. Nếu bạn đưa robot tới bằng tay và nó lệch 1 mm so với vị trí lý tưởng, thì mọi lần chạy về sau sẽ lặp lại đúng 1 mm lệch ấy. Đây không phải nhược điểm mà là đặc tính: mero cho bạn độ lặp lại, không cho độ chính xác. Nếu vị trí đích được xác định bởi chính cơ khí — robot cập vào, chốt khớp, tiếp điểm chạm — thì lần dạy tự nó chính xác và vấn đề biến mất.

CẦN NHỚ

Teach-by-showing biến bài toán từ *accuracy* thành *repeatability*, và đó là một thay đổi bậc độ lớn chứ không phải một cải thiện tăng dần — vì trong một hệ điển hình, các nguồn hệ thống chiếm phần lớn ngân sách.

Ba điều kiện để nó hoạt động:

Một, g phải không đổi giữa lần dạy và lần chạy — nên phải khoá mọi thứ khoá được (exposure, gain, nét, vị trí camera) và dạy lại định kỳ.

Hai, robot phải tới gần tư thế dạy — triệt tiêu là ở bậc nhất, nên nó tốt ở đoạn cuối và kém ở đoạn đầu. Đây là lý do coarse-to-fine ở mục 5 là bạn đồng hành tự nhiên của mero này.

Ba, lần dạy phải đúng — lý tưởng nhất là để cơ khí xác định vị trí đích thay vì mắt người.

Nếu bài toán của bạn đúng là “về đúng một chỗ”, đây là con đường ngắn nhất tới 5 mm, và nó nên là quyết định kiến trúc đầu tiên chứ không phải tối ưu hoá cuối cùng.

1.4 Stop-and-go

Mục ngắn, vì A/06 đã dẫn cơ sở. Nó trả lời Q4.

Dừng hẳn robot, chụp N khung, tính trung bình, rồi bước tiếp. Hai lợi ích, và chúng khác nhau về bản chất — đó là phần thứ hai của Q4.

Lợi ích thứ nhất, giảm nhiễu: trung bình N khung giảm thành phần ngẫu nhiên theo $1/\sqrt{N}$. Với $N = 30$, đó là hệ số 5,5. Đây là lợi ích ai cũng biết.

Lợi ích thứ hai, xoá thiên lệch: rolling shutter và motion blur là các nguồn hệ thống tỉ lệ với vận tốc, nên ở $v = 0$ chúng bằng không (A/06 mục 7). Đây là lợi ích lớn hơn, vì trung bình nhiều khung

không bao giờ chạm tới được các nguồn hệ thống. Nói cách khác: trung bình giảm một dòng ngân sách theo $\sqrt{N}$, còn dừng lại thì *xoá hẳn* hai dòng.

Chi phí: thời gian. Dừng 0,3 giây mỗi lần đo, với vài lần đo ở đoạn cuối, là vài giây cho cả quy trình cập dock — gần như luôn chấp nhận được trong ứng dụng docking, vì đây không phải thao tác thời gian thực gấp gáp. (Con số 0,3 giây và 30 khung là ví dụ minh họa; số đúng phụ thuộc tốc độ khung hình và thời gian ổn định cơ khí sau khi phanh — đừng quên rằng robot còn rung một lúc sau khi bánh xe dừng.)

Chi tiết dễ quên vừa nêu đáng nhấn mạnh: *phải đợi rung tắt*. Dừng bánh xe không có nghĩa là camera đứng yên; thân robot còn dao động, và chụp ngay sau khi phanh thì bạn vẫn có motion blur. Cách kiểm: đo độ tản mát của s theo thời gian sau khi phanh, và chọn thời điểm bắt đầu chụp là lúc độ tản mát ấy chạm sàn.

1.5 Coarse-to-fine

Nguyên tắc tổ chức của cả hành trình, và nó là cái khung mà bốn công cụ kia gắn vào.

Quan sát: *độ chính xác cần thiết không đồng đều theo hành trình*. Ở 3 m, robot chỉ cần biết hướng đại khái để tiến về phía dock — sai 10 cm không sao. Ở 0,5 m, nó cần 5 mm. Yêu cầu tăng hai bậc độ lớn, và may mắn là khả năng cũng tăng theo cùng chiều: $A/05$ cho $\delta x \approx Z\sigma/f$, tuyến tính theo Z , nên tới gần thì tự nhiên chính xác hơn.

Chia hành trình thành ba pha, mỗi pha một cấu hình khác nhau.

Pha xa (3 m tới 1 m). Dùng tag lớn ở ngoại vi (B/09 mục 6). Điều khiển liên tục, không dừng. Mục tiêu: đưa robot vào vùng mà cụm tag nhỏ nhìn thấy được và vào gần trực tiếp cận. Dung sai rộng, nên rolling shutter và blur không quan trọng.

Pha gần (1 m tới 0,3 m). Chuyển sang cụm tag nhỏ ở trung tâm. Vẫn điều khiển liên tục nhưng chậm hơn. Bắt đầu bật các phép kiểm nhất quán của B/09 mục 7.

Pha cuối (0,3 m tới chạm). Stop-and-go, trung bình nhiều khung, teach-by-showing với sai lệch tương đối. Đây là pha duy nhất mà ngân sách 5 mm áp dụng, và mọi kỹ thuật đắt tiền đều dồn vào đây.

Cấu trúc này có một lợi ích không hiển nhiên: nó cho phép các pha dùng *tiêu chí khác nhau*. Pha xa có thể dùng PBVS vì nó cần quỹ đạo dự đoán được để tránh vật cản; pha cuối có thể dùng IBVS vì nó cần độ bền với hiệu chuẩn và không cần quỹ đạo phức tạp. Chuyển đổi giữa hai luật điều khiển ở một điểm mà cả hai đều hợp lệ là một kỹ thuật chuẩn và nó đơn giản hơn nhiều so với việc ép một luật duy nhất phục vụ cả hai mục đích.

1.6 Ràng buộc nonholonomic

Mục này trả lời Q3, và nó là chỗ bài toán docking của robot di động khác hẳn bài toán servoing của cánh tay máy.

Robot vì sai hai bánh có ba bậc tự do cấu hình (x, y, θ) nhưng chỉ hai đầu vào điều khiển (vận tốc tiến v và vận tốc quay ω). Nó *không đi ngang được*: vận tốc theo phương vuông góc trục bánh luôn bằng không. Đây là ràng buộc nonholonomic — ràng buộc trên vận tốc, không trên vị trí.

Hệ quả thứ nhất, dễ thấy: sửa một lệch ngang 2 cm đòi một cơ động — tiến, quay, lùi, quay — chứ không phải một chuyển động thẳng. Với không gian chật trước dock, cơ động ấy có thể không thực hiện được.

Hệ quả thứ hai, sâu hơn và đây là phần thứ hai của Q3: *định lý Brockett* nói rằng không tồn tại luật phản hồi trạng thái trơn, dừng (không phụ thuộc thời gian) nào ổn định hoá hệ nonholonomic về một điểm [7]. Đây không phải một khó khăn kỹ thuật mà là một bất khả về cấu trúc: bất kỳ ai hứa hẹn một bộ điều khiển tỉ lệ đơn giản đưa robot vi sai về đúng một tư thế đều đang bỏ sót điều gì đó.

Lỗi ra có ba, và cả ba đều được dùng trong thực tế.

Luật điều khiển không trơn hoặc phụ thuộc thời gian. Định lý Brockett chỉ loại các luật trơn và dừng; nói một trong hai điều kiện thì có lời giải [8]. Đây là con đường lý thuyết đúng và nó có các luật đã được chứng minh.

Thiết kế đường tiếp cận thẳng trục. Đây là con đường thực dụng và tôi khuyên nó. Thay vì cố sửa lệch ngang ở gần dock, hãy *đảm bảo không có lệch ngang* bằng cách đưa robot vào một đoạn tiếp cận thẳng từ xa: ở khoảng 1–1,5 m, robot căn chỉnh để nằm trên trục dock, rồi từ đó chỉ đi thẳng. Sai lệch ngang được sửa ở nơi có không gian để cơ động, và pha cuối chỉ còn một bậc tự do cần điều khiển chặt. Điều này đồng thời giúp cho thị giác: đi thẳng trục nghĩa là góc nhìn tag không đổi nhiều, nên teach-by-showing triệt tiêu tốt.

Cơ khí. Mục 7.

Đáng nêu một đánh đổi mà mục này tạo ra với A/05: đi thẳng trục là tư thế fronto-parallel, tức tư thế *tệ nhất* cho việc đo góc từ một tag phẳng. Hai yêu cầu xung đột. Cách giải quyết đúng không phải là hy sinh một trong hai mà là *ngiên các tag* (B/09 mục 4): robot đi thẳng trục, nhưng các tag không vuông góc với trục ấy. Đây là một ví dụ đẹp của việc một quyết định cơ khí giải quyết một xung đột mà không quyết định phần mềm nào giải quyết được.

1.7 Dung sai cơ khí: côn dẫn hướng

Mục này trả lời Q5, và nó là chỗ một cây tri thức về thị giác phải trung thực thừa nhận rằng đôi khi câu trả lời không nằm ở thị giác.

Ý tưởng: thêm một mặt vát (chamfer) hoặc một côn dẫn hướng ở miệng dock. Khi robot tới với một sai lệch nhỏ, đầu cảm chạm vào mặt vát và được *dẫn* vào đúng chỗ thay vì va vào mép. Lý thuyết của chuyện này thuộc ngành lắp ráp tự động và đã có từ lâu; Whitney phân tích điều kiện để một phép lắp có tuân thủ thành công dưới sai lệch vị trí và góc [3].

Hiệu quả về mặt yêu cầu: nếu côn dẫn hướng có bề rộng vào w_c , thì yêu cầu đối với hệ thị giác giảm từ “dưới dung sai khớp” xuống “dưới w_c ”. Với một côn rộng 15 mm, bài toán 5 mm thành bài toán 15 mm — và theo A/05, đó là một bài toán dễ hơn ba lần theo mọi chiều, tức dễ hơn nhiều lần về công sức.

Vì sao đề xuất ấy thường đúng: chi phí. Một mặt vát trên một chi tiết đã phải gia công là gần như miễn phí. So với nó, việc hạ sai số thị giác từ 15 mm xuống 5 mm tốn hiệu chuẩn kỹ hơn, constellation chính xác hơn, có thể là camera tốt hơn, và nhiều tuần kỹ thuật.

Ba trường hợp nó *không* giúp, và đây là phần thứ hai của Q5.

Một, khi sai lệch góc là vấn đề chứ không phải sai lệch vị trí. Côn dẫn hướng sửa được lệch vị trí tốt; sửa lệch góc thì kém hơn nhiều, vì một đầu cảm nghiêng có thể kẹt thay vì trượt vào. Với ngân sách $0,5^\circ$, côn giúp ít.

Hai, khi tiếp điểm cần chính xác hơn cơ khí dẫn được. Sạc điện thì côn giúp; cắm một đầu nối quang học đòi căn chỉnh vi mét thì không.

Ba, khi không có lực để dẫn. Côn hoạt động nhờ robot tiếp tục đẩy tới và lực phản từ mặt vát đẩy nó vào đúng chỗ. Nếu robot nhẹ, hoặc nếu bánh trượt, hoặc nếu cơ cấu không cho phép tuân thủ,

thì thay vì được dẫn vào, robot sẽ bị chặn lại. Điều này dẫn tới một yêu cầu thiết kế đi kèm: phải có *độ tuân thủ* (compliance) ở đâu đó trong chuỗi — lò xo, cao su, hoặc một cơ cấu nổi.

CẦN NHỚ

Trước khi bỏ công hạ sai số thì giác một bậc, hãy hỏi kỹ sư cơ khí xem có thể nói dung sai một bậc không.

Thường thì có, và thường thì nó rẻ hơn nhiều. Đây không phải một lời khuyên khiêm tốn giả tạo — nó là kết luận đúng của một phân tích chi phí, và một kỹ sư thì giác không đặt câu hỏi ấy là đang bỏ sót một biến trong bài toán tối ưu.

Nhưng có một điều kiện: còn dẫn hướng sửa *vị trí* tốt hơn sửa *góc* nhiều. Với ngân sách $0,5^\circ$, nó giúp ít — và theo A/05 thì góc vốn đã là nút thắt. Nên cách đọc đúng là: cơ khí nói được dòng ngân sách tịnh tiến (vốn đã dư), và không nói được dòng ngân sách góc (vốn đang thiếu).

Trừ khi — và đây là chỗ đáng nghĩ — cơ cấu được thiết kế để *tự căn góc*: một đôi chốt còn thay vì một chốt duy nhất, hoặc một mặt phẳng tựa. Khi ấy nó nói được cả hai, và đó là đề xuất đáng đưa ra với kỹ sư cơ khí.

1.8 Tiêu chí hội tụ và cơ chế retry

Chi tiết kỹ thuật cuối, và nó là chỗ hay bị làm ầu.

Bộ điều khiển cần một tiêu chí dừng. Tiêu chí sai lầm phổ biến: dừng khi sai lệch nhỏ hơn ngưỡng. Vấn đề là nó không phân biệt “sai lệch nhỏ vì đã tới đích” với “sai lệch nhỏ vì phép đo sai”.

Tiêu chí đúng gồm ba điều kiện, phải thoả đồng thời.

Một, sai lệch dưới ngưỡng — điều kiện hiển nhiên.

Hai, phép đo đáng tin — số tag nhìn thấy đủ, tỉ số ambiguity ρ trên ngưỡng ($B/10$), sai số tái chiếu trong khoảng bình thường, phép kiểm nhất quán liên tag ($B/09$ mục 7) không báo động, và hiệp phương sai từ $\mathcal{I}^{-1}$ đủ nhỏ.

Ba, ổn định — sai lệch giữ dưới ngưỡng qua nhiều khung liên tiếp, chứ không phải một khung may mắn.

Khi một trong ba điều kiện không thoả sau một thời gian chờ, cần một cơ chế *retry* chứ không phải báo lỗi. Retry hợp lý: lùi lại một khoảng, đợi, tiếp cận lại. Lý do nó thường hiệu quả: nhiều nguyên nhân thất bại là tạm thời hoặc phụ thuộc tư thế — một vật nặng chiếu vào tag, một góc nhìn xấu làm ambiguity nặng, một vật cản thoáng qua — và tiếp cận lại từ một điểm hơi khác thường thoát ra được.

Ghi lại lý do mỗi lần retry vào log. Phân bố của các lý do ấy sau một tháng vận hành là dữ liệu chẩn đoán giá trị nhất bạn có thể có về hệ thống của mình, và nó thường chỉ ra nút thắt thật — thứ mà không phân tích lý thuyết nào đoán trước được.

1.9 Giới hạn và chế độ hỏng

Teach-by-showing đòi một lần dạy đúng. Đã bàn ở mục 3C. Nếu vị trí đích không được cơ khí xác định thì độ chính xác tuyệt đối của hệ bị giới hạn bởi chất lượng lần dạy, và mọi lần chạy sẽ lặp lại sai lệch ấy một cách trung thành.

Teach-by-showing không xử lý được thay đổi. Nếu dock bị dịch chuyển hoặc tag bị thay, phải dạy lại. Đây là đánh đổi so với PBVS đầy đủ: PBVS hoạt động ở bất kỳ dock nào có constellation đã biết; teach-by-showing gắn với một dock cụ thể.

IBVS có bệnh lý. Camera retreat và cực tiểu cục bộ trong không gian ảnh. Với hình học đơn giản của docking thì hiếm, nhưng phải kiểm bằng mô phỏng toàn bộ dải tư thế ban đầu chứ không chỉ vài trường hợp.

Định lý Brockett không tránh được. Mọi lời hứa về một bộ điều khiển tỉ lệ tron đưa robot vi sai về đúng một tư thế đều sai [7]. Nếu một cài đặt trông như đang làm được điều đó, thì hoặc nó không thực sự hội tụ về tư thế (chỉ về vị trí), hoặc nó đang dùng một luật không tron mà tác giả không nhận ra.

Các con số trong chương là ví dụ. Côn 15 mm, dừng 0,3 giây, 30 khung, chuyển pha ở 1 m và 0,3 m — tất cả là ví dụ minh họa do tôi đặt để có một cấu hình cụ thể nói chuyện. Con số đúng cho hệ của bạn phụ thuộc cơ khí, camera, và tốc độ khung hình.

I 1.10 Thực hành

Năm việc, theo thứ tự ưu tiên — và thứ tự này khác với thứ tự trình bày trong chương, có chủ ý. Một, *hỏi kỹ sư cơ khí trước*. Có côn dẫn hướng được không? Có tự căn góc được không? Câu trả lời có thể làm nhẹ toàn bộ phần còn lại.

Hai, *quyết định teach-by-showing hay không, ngay từ đầu*. Đây là quyết định kiến trúc, và nó ảnh hưởng tới việc bạn cần hiệu chuẩn kỹ tới mức nào ở B/12. Nếu bài toán là “về đúng một chỗ” thì chọn nó.

Ba, *thiết kế đường tiếp cận thẳng trục*, với việc căn chỉnh ngang hoàn tất ở 1–1,5 m. Và nhớ bù lại bằng cách nghiêng tag (B/09 mục 4), vì đi thẳng trục là tư thế xấu cho đo góc.

Bốn, *stop-and-go ở pha cuối*, với thời gian đợi rung tắt được đo chứ không đoán.

Năm, *tiêu chí hội tụ ba điều kiện và cơ chế retry có ghi log lý do*.

I Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Quan sát trung tâm của chương	Docking cần <i>repeatability</i> , không cần <i>accuracy</i> [1]. Robot không cần biết nó ở đâu; nó cần về đúng chỗ nó đã từng ở.
••• Vì sao teach-by-showing triệt tiêu sai số hệ thống	Cùng một hàm sai lệch g tác động lên cả pose đo hiện tại lẫn pose đo mục tiêu; khi bộ điều khiển làm hai cái bằng nhau thì hai pose <i>thật</i> bằng nhau, bất kể g là gì — miễn g đơn ánh và không đổi.
••• Bốn chỗ triệt tiêu không hoàn toàn	g phụ thuộc tư thế (chỉ triệt tiêu ở bậc nhất, tốt khi gần đích); g trôi theo thời gian (dạy lại định kỳ); nguồn ngẫu nhiên không triệt tiêu; và sai số của chính lần dạy được lặp lại trung thành.
••• Hai vấn đề IBVS tránh hoàn toàn	Pose ambiguity (không giải PnP nên không phải chọn nghiệm); và độ nhạy với hiệu chuẩn — K sai chỉ vào qua ma trận tương tác L , ảnh hưởng <i>đường đi</i> chứ không ảnh hưởng <i>điểm dừng</i> [4].
•• Luật IBVS cơ bản	$\mathbf{v} = -\lambda L^+ \mathbf{e}$ với $\mathbf{e} = \mathbf{s} - \mathbf{s}^*$ là sai lệch trong không gian ảnh [2, §II].
•• Cái giá của IBVS	Quy đạo 3D không kiểm soát trực tiếp; bệnh lý camera retreat khi có phép xoay lớn quanh trục quang. Hiếm với hình học docking nhưng phải kiểm bằng mô phỏng.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Hai lợi ích khác bản chất của stop-and-go	Giảm nhiễu theo $1/\sqrt{N}$ (ai cũng biết); và <i>xoá</i> rolling shutter cùng motion blur vì chúng tỉ lệ với vận tốc (lớn hơn, vì trung bình không bao giờ chạm tới nguồn hệ thống).
●● Chi tiết dễ quên của stop-and-go	Phải đợi <i>rung tắt</i> , không phải chỉ đợi bánh xe dừng. Đo bằng cách theo dõi độ tần mất của s sau khi phanh và chọn thời điểm nó chạm sàn.
●●● Định lý Brockett	Không tồn tại luật phản hồi trạng thái <i>trơn và dừng</i> nào ổn định hoá hệ nonholonomic về một điểm [7]. Bất khả về cấu trúc, không phải khó khăn kỹ thuật.
●●● Lỗi ra thực dụng cho nonholonomic	Thiết kế đường tiếp cận thẳng trục: căn chỉnh ngang xong ở 1–1,5 m nơi có không gian cơ động, rồi chỉ đi thẳng. Pha cuối chỉ còn một bậc tự do cần điều khiển chặt.
●●● Xung đột giữa mục 6 và A/05	Đi thẳng trục là tư thế fronto-parallel — tệ nhất cho đo góc. Cách giải: <i>ngiên tag</i> chứ không nghiêng quỹ đạo. Một quyết định cơ khí giải quyết xung đột mà phần mềm không giải được.
●●● Côn dẫn hướng biến bài toán 5 mm thành bài toán w_c mm	Chi phí gần bằng không trên chi tiết đã phải gia công [3]. Nhưng nó sửa <i>vị trí</i> tốt hơn sửa <i>góc</i> nhiều — nên nó nói dòng tịnh tiến (vốn đã dư) chứ không nói dòng góc (vốn đang thiếu).
●● Ba trường hợp côn không giúp	Khi sai lệch <i>góc</i> là vấn đề; khi tiếp điểm cần chính xác hơn cơ khí dẫn được; khi không có lực hoặc không có độ tuân thủ để dẫn.
●● Ba pha coarse-to-fine	Xa: tag lớn, điều khiển liên tục, dung sai rộng. Gần: tag nhỏ, chậm hơn, bật kiểm nhất quán. Cuối: stop-and-go, trung bình nhiều khung, teach-by-showing. Các pha có thể dùng <i>luật điều khiển khác nhau</i> .
●●● Tiêu chí hội tụ ba điều kiện	Sai lệch dưới ngưỡng và phép đo đáng tin (đủ tag, ρ cao, residual bình thường, nhất quán liên tag, hiệp phương sai nhỏ) và ổn định qua nhiều khung. Chỉ điều kiện thứ nhất là không phân biệt được “tới đích” với “đo sai”.
● Giá trị của log lý do retry	Phân bố lý do sau một tháng vận hành là dữ liệu chẩn đoán giá trị nhất về hệ thống, và nó thường chỉ ra nút thắt thật — thứ không phân tích lý thuyết nào đoán trước được.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ViSP [9]	Thư viện visual servoing đầy đủ: IBVS, PBVS, lai, có sẵn tracker marker và tính ma trận tương tác	Nơi mục 2 thành code; đọc phần IBVS kèm [2]
Bài gốc IBVS [4]	Nguồn của luận điểm “điểm dừng không phụ thuộc hiệu chuẩn”	Đọc để hiểu vì sao IBVS bền, không chỉ biết rằng nó bền
Tutorial Chaumette–Hutchinson [2], [6]	Tài liệu chuẩn hai phần cho toàn bộ mục 2, gồm cả các trường hợp hỏng	Phần II bàn chính các bệnh lý ở mục 9
Lắp ráp có tuân thủ [3]	Lý thuyết chamfer và điều kiện thành công dưới sai lệch vị trí/góc	Nguồn cho mục 7; đáng đưa cho kỹ sư cơ khí
Điều khiển hệ xích [8]	Luật điều khiển phụ thuộc thời gian cho hệ nonholonomic, lách định lý Brockett	Con đường lý thuyết đúng nếu không thể thiết kế đường tiếp cận thẳng trục

MINI PROJECT · Đo lợi ích thật của teach-by-showing trên hệ của bạn · 3–4 ngày

Mục tiêu	biến luận điểm trung tâm của chương từ một lập luận thành một cặp số đo, và biết chính xác mẹo này đáng giá bao nhiêu milimét trong hệ cụ thể của bạn.
Dữ liệu	phần cứng thật. Cần một cách xác định vị trí đích độc lập với thị giác — lý tưởng nhất là chính cơ cấu dock (robot cập vào, chốt khớp), hoặc một đồ gá cơ khí có dung sai biết trước.
Các bước	giai đoạn một, đo độ <i>chính xác</i> : cố ý làm hỏng hiệu chuẩn một chút (dịch c_x đi 10 px, hoặc dùng tọa độ constellation từ CAD thay vì từ BA), rồi cho robot cập dock theo PBVS tuyệt đối 20 lần, đo sai lệch cuối bằng chuẩn cơ khí; giai đoạn hai, đo độ <i>lặp lại</i> : với đúng hiệu chuẩn hỏng ấy, dạy một lần rồi cho cập 20 lần theo teach-by-showing, đo lại; giai đoạn ba, lặp cả hai với hiệu chuẩn đúng; giai đoạn bốn, lặp giai đoạn hai nhưng cho robot xuất phát từ các vị trí ban đầu xa dần từ thể dạy, để đo tốc độ mất hiệu lực của phép triệt tiêu bậc nhất.
Xong khi	bạn có bốn con số: accuracy với hiệu chuẩn hỏng, repeatability với hiệu chuẩn hỏng, và hai con số tương ứng với hiệu chuẩn đúng. Nếu lập luận của mục 3 đúng thì con số thứ hai phải gần con số thứ tư — tức teach-by-showing làm cho việc hiệu chuẩn hỏng gần như không quan trọng. Từ giai đoạn bốn bạn có một đường cong nói mẹo mất hiệu lực ở khoảng cách nào, và đó là con số quyết định pha cuối của coarse-to-fine nên bắt đầu ở đâu. Kiểm tra chéo bắt buộc: nếu repeatability với hiệu chuẩn hỏng <i>không</i> tốt hơn accuracy đáng kể, thì hoặc sai số bạn cố ý thêm vào không phải sai số hệ thống thuần (kiểm lại), hoặc ngân sách của bạn đang bị chi phối bởi nguồn ngẫu nhiên chứ không phải hệ thống — và đó cũng là một kết quả quan trọng, nó nói bạn nên tập trung vào B/11 thay vì B/12.
Nối với	C/15 nhận bốn con số này để hiệu chỉnh bảng ngân sách; C/16 mở rộng thành quy trình nghiệm thu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Teach-by-showing: ghi lại trạng thái cảm biến tại vị trí cập chuẩn một lần, rồi điều khiển để tái tạo đúng trạng thái ấy thay vì điều khiển tới một tọa độ tính toán. Cơ chế: nếu g là hàm biểu diễn toàn bộ sai số hệ thống, thì pose đo được là g của pose thật ở cả hai vế; khi bộ điều khiển làm hai pose đo bằng nhau và g đơn ánh, thì hai pose *thật* bằng nhau — bất kể g là gì, và không cần biết nó. Nó “gần như” chứ không “hoàn toàn” vì bốn lý do: g thực ra phụ thuộc tư thế (méo còn sót, bias ellipse, điểm chính — nên triệt tiêu chỉ ở bậc nhất quanh tư thế dạy và kém dần khi ra xa); g trôi theo thời gian (nhiệt, bẩn, va chạm — cần dạy lại định kỳ); nguồn ngẫu nhiên không bị ảnh hưởng gì; và sai lệch của chính lần dạy được lặp lại trung thành ở mọi lần chạy.
- A2. Vấn đề thứ nhất là pose ambiguity: IBVS không giải PnP nên không bao giờ phải chọn giữa hai nghiệm, và toàn bộ A/04 cùng B/10 trở nên không liên quan. Vấn đề thứ hai là độ nhạy với hiệu chuẩn: sai số của K và của constellation chỉ vào qua ma trận tương tác L , tức chỉ ảnh hưởng tới hướng và độ lớn của *bước điều khiển*, không ảnh hưởng tới *điểm dừng* — vì điểm dừng được định nghĩa bởi $e = s - s^* = 0$, một điều kiện thuần túy trên ảnh không chứa K [4]. Hệ quả: một L sai làm robot đi vòng vèo hơn nhưng vẫn tới đúng đích. Cái giá là quỹ đạo trong không gian ba chiều không được kiểm soát trực tiếp, nên có thể xuất hiện các chuyển động kỳ lạ như camera retreat — hiểm với hình học docking đơn giản nhưng phải kiểm bằng mô phỏng trên toàn dải tư thế ban đầu.
- A3. Khó hơn vì robot vi sai có ràng buộc nonholonomic: vận tốc theo phương vuông góc trục bánh luôn bằng không, nên nó không đi ngang được. Sửa 2 cm ngang đòi một cơ động nhiều bước — tiến, quay, lùi, quay — trong khi sửa 2 cm tiến lùi chỉ là một chuyển động thẳng; và trước dock thường không có không gian cho cơ động ấy. Định lý Brockett [7] nói rằng không tồn tại luật phản hồi trạng thái *trơn và dừng* nào ổn định hoá một hệ nonholonomic về một điểm — đây là bất khả về cấu trúc chứ không phải khó khăn kỹ thuật, nên bất kỳ ai hứa một bộ điều khiển tỉ lệ đơn giản làm được việc đó đều đang bỏ sót gì đó. Lỗi ra thực dụng nhất không phải là tìm luật khéo hơn mà là *thiết kế đường tiếp cận thẳng trực*: hoàn tất căn chỉnh ngang ở 1–1,5 m nơi còn không gian, rồi pha cuối chỉ đi thẳng.

- A4. Lợi ích thứ nhất là giảm *nhiều*: trung bình N khung giảm thành phần ngẫu nhiên theo $1/\sqrt{N}$, với $N = 30$ là hệ số 5,5. Lợi ích thứ hai là xoá *thiên lệch*: rolling shutter và motion blur đều tỉ lệ với vận tốc nên bằng không ở $v = 0$ (A/06 mục 7). Lợi ích thứ hai lớn hơn, và lớn hơn theo một cách có tính nguyên tắc: lấy trung bình *không bao giờ* chạm tới được các nguồn hệ thống, dù lấy bao nhiêu khung đi nữa, nên việc xoá hẳn hai dòng ngân sách có giá trị hơn việc chia một dòng khác cho 5,5. Một chi tiết thực hành hay bị quên: phải đợi *rung tắt* chứ không chỉ đợi bánh xe dừng, vì thân robot còn dao động sau khi phanh.
- A5. Thường đúng vì chi phí: một mặt vát trên một chi tiết vốn đã phải gia công là gần như miễn phí, còn việc hạ sai số thị giác từ 15 mm xuống 5 mm tốn hiệu chuẩn kỹ hơn, constellation chính xác hơn, có thể cả camera tốt hơn, cộng nhiều tuần kỹ thuật [3]. Ba trường hợp nó không giúp. Một, khi vấn đề là sai lệch góc: còn dẫn hướng sửa lệch vị trí tốt nhưng một đầu cảm nghiêng thì kẹt chứ không trượt vào — và theo A/05 thì góc mới là nút thắt, nên đây là hạn chế đáng kể. Hai, khi tiếp điểm đòi chính xác hơn mức cơ khí dẫn được. Ba, khi không có lực hoặc không có độ tuân thủ để dẫn: còn hoạt động nhờ robot tiếp tục đẩy và lực phản từ mặt vát đẩy nó vào chỗ, nên cần khối lượng, ma sát bánh đủ, và một chỗ nào đó trong chuỗi có thể nhường — lò xo, cao su, cơ cấu nổi. Đề xuất đáng đưa ngược lại cho kỹ sư cơ khí: thiết kế cơ cấu *tự căn góc* (đôi chốt côn, hoặc mặt phẳng tựa) thay vì chỉ dẫn vị trí, vì như vậy nó nối được cả dòng ngân sách đang thiếu.

NÓI LẠI THẬT ĐƠN GIẢN

Có một câu hỏi nên hỏi trước khi bắt tay vào làm cho hệ thống chính xác hơn: *robot có thật sự cần biết nó đang ở đâu không?*

Nghĩ về việc bạn tra chìa vào ổ khoá nhà mình trong bóng tối. Bạn không đo khoảng cách, không tính toạ độ. Bạn làm đúng cái động tác bạn đã làm hàng nghìn lần, và tay bạn biết. Nếu ai đó hỏi cái ổ khoá cách mép cửa bao nhiêu xăng-ti-mét thì bạn chịu — và điều đó chẳng ảnh hưởng gì tới việc bạn mở được cửa.

Robot cập dock cũng vậy. Nó không cần biết toạ độ. Nó cần về đúng chỗ nó đã từng ở. Và hai việc ấy khác nhau nhiều hơn về ngoài.

Đây là chỗ hay: nếu ta đưa robot tới đúng vị trí cập chuẩn một lần — đẩy bằng tay cũng được — rồi chụp lại xem camera nhìn thấy gì, thì về sau việc của robot chỉ là lái sao cho nhìn thấy đúng như vậy. Và bây giờ mọi sai lệch cố định của hệ thống — ống kính hơi lệch, tấm marker hơi cong, camera gấn hơi nghiêng — đều tự biến mất. Vì sao? Vì chúng làm hình ảnh lệch đi đúng như vậy ở *cả hai lần*: lần chụp mẫu và lần đang cập. Hai cái lệch giống nhau thì trừ nhau ra không.

Bạn không cần biết chúng là gì. Bạn không cần đo chúng. Bạn chỉ cần chúng dừng thay đổi.

Cái “dừng thay đổi” ấy là chỗ duy nhất phải cẩn thận. Nếu ống kính nóng lên và giãn ra, nếu tấm marker bị va, nếu ai đó vặn vòng lấy nét — thì hình ảnh mẫu không còn khớp nữa. Cách phòng đơn giản: dán keo vào những gì vặn được, và thỉnh thoảng chụp lại ảnh mẫu.

Còn một mẹo thứ hai, đơn giản đến mức dễ bỏ qua: *dừng lại trước khi nhìn*. Khi robot đang chạy, ảnh bị nhoè và — với nhiều loại camera — còn bị xé nữa, vì chúng quét ảnh từ trên xuống chứ không chụp cả tấm cùng lúc. Cả hai chuyện đó biến mất nếu robot đứng yên. Dừng một phần ba giây, chụp ba chục tấm, lấy trung bình, rồi đi tiếp. Không tốn tiền, không tốn code.

Và mẹo thứ ba, mẹo mà một người làm thị giác hay ngại nói ra: *hỏi anh cơ khí*. Nếu miệng dock được vát một cái phễu rộng một phân rưỡi, thì robot chỉ cần vào đúng trong phạm vi một phân rưỡi là cái phễu tự đẩy nó vào chỗ. Bài toán năm milimét thành bài toán mười lăm milimét, và cái phễu ấy tốn gần như không có gì. Đây không phải thua cuộc — đây là chọn đúng chỗ để tiêu tiền.

Chỗ không rút gọn được: cái mẹo “chụp lại ảnh mẫu” chỉ hoạt động tốt khi robot đã tới gần chỗ nó chụp mẫu. Càng xa thì hiệu quả càng kém, và kém theo một kiểu trơn tru chứ không có ranh giới rõ. Xa bao nhiêu thì bắt đầu kém — cái đó phải đo trên chính hệ của mình.

Một câu: Dừng dạy robot biết nó ở đâu; dạy nó nhận ra chỗ nó đã từng đứng.

1.11 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *teach-by-showing* triệt tiêu sai số hệ thống và biến bài toán accuracy thành bài toán repeatability — bị bác bỏ nếu mini project cho thấy độ lặp lại với hiệu chuẩn cố ý làm hỏng không tốt hơn đáng kể so với độ chính xác với cùng hiệu chuẩn ấy. Tôi dẫn mệnh đề bằng lập luận đại số ở mục 3B (cửa a) và đối chiếu với luận điểm tương đương về IBVS trong [4] (cửa c một phần). Nó chưa qua cửa (b), và đây là nợ kiểm chứng lớn nhất của chương — cũng là mệnh đề có hậu quả kiến trúc lớn nhất trong cả cây, nên nên kiểm sớm nhất có thể.

Mệnh đề thứ hai — *triệt tiêu chỉ ở bậc nhất, nên hiệu quả giảm khi xa tư thế dạy* — bị bác bỏ nếu giai đoạn bốn của mini project cho thấy độ lặp lại không xấu đi khi xuất phát từ các vị trí ban đầu xa dần. Nếu vậy thì lời khuyên ghép teach-by-showing với coarse-to-fine là thừa, và mẹo dùng được rộng hơn tôi nghĩ.

Mệnh đề thứ ba — *côn dẫn hướng sửa vị trí tốt hơn sửa góc* — là suy luận hình học của tôi và nó bị bác bỏ nếu một thí nghiệm cơ khí cho thấy một cơ cấu côn thông thường cũng dẫn được sai lệch góc tới cùng mức. Điều đáng nói: nếu nó bị bác bỏ thì đó là *tin tốt*, vì khi ấy cơ khí nói được cả dòng ngân sách đang thiếu.

Cuối cùng, một mệnh đề mà chương này ngầm giả định và đáng nêu ra: *bài toán của bạn thật sự là lặp lại một vị trí*. Nếu robot phải cập vào nhiều dock khác nhau, hoặc dock có thể bị dịch chuyển, hoặc phải cập vào một vị trí được chỉ định động, thì teach-by-showing không áp dụng được và bạn quay về bài toán accuracy đầy đủ — tức quay về Phần A và B/12 với mọi yêu cầu khắt khe của chúng. Chương này không giấu điều đó, nhưng nó đáng được nói thành lời.

1.12 Kết nối

Chương này là chỗ toàn bộ cây trả lại giá trị thực dụng, và nó có một tính chất đặc biệt: nó *làm nhẹ* các chương khác thay vì chồng thêm yêu cầu.

Nói lên trên. A/05-lan-truyen-sai-so-pixel-sang-pose mục 8 cấp toàn bộ nền lý thuyết cho mục 3: phân loại nguồn ngẫu nhiên và hệ thống là điều kiện để hiểu vì sao teach-by-showing hoạt động, và cũng là điều kiện để hiểu giới hạn của nó. A/06-hieu-ung-vat-ly-va-quang-hoc mục 7 cấp cơ sở vật lý cho mục 4. A/04-pose-ambiguity-target-phang là chương mà IBVS ở mục 2 làm cho không liên quan — một mối quan hệ hiếm trong cây: một chương làm một chương khác không cần thiết.

Nói ngang trong Phần B, và đây là phần quan trọng nhất về mặt tổ chức dự án. B/12-calibration-cho-docking là chương mà chương này *nói lỏng* nhiều nhất: nếu dùng teach-by-showing thì yêu cầu về độ chính xác tuyệt đối của intrinsic và hand-eye giảm mạnh, vì chúng là nguồn hệ thống và chúng triệt tiêu. Điều còn lại cần thiết là *tính ổn định* của hiệu chuẩn, không phải độ đúng của nó — một yêu cầu dễ hơn nhiều. B/09-multi-marker-constellation cũng được nói: sai số constellation là nguồn hệ thống. Nhưng — và đây là chỗ tinh tế — nó chỉ được nói cho phần *đo*, không cho phần *hình học*: baseline và tính không đồng phẳng vẫn cần thiết nguyên vẹn, vì chúng quyết định σ hiệu dụng chứ không phải bias. B/11-uoc-luong-theo-thoi-gian trở nên *quan trọng hơn* sau chương này, vì khi các nguồn hệ thống đã bị triệt tiêu thì các nguồn ngẫu nhiên trở thành phần còn lại duy nhất, và chúng chính là thứ mà trung bình theo thời gian xử lý.

Nói xuống Phần C. C/15-ngan-sach-sai-so phải có *hai* bảng ngân sách chứ không một: một cho chế độ accuracy tuyệt đối, một cho chế độ teach-by-showing. Chênh lệch giữa hai bảng ấy là giá trị định lượng của chương này, và nó thường là một hệ số lớn. C/16-danh-gia-va-nghiem-thu nhận từ mục 3 một yêu cầu phương pháp luận quan trọng: phải đo và báo cáo *riêng* accuracy và repeatability [1], vì gộp chúng lại thì không ai biết hệ thống thật sự làm được gì.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao độ lặp lại là một yêu cầu dễ hơn độ chính xác một bậc, và nguồn sai số nào tạo ra sự chênh lệch ấy?
2. Dẫn lại trong ba dòng vì sao teach-by-showing đưa robot tới đúng vị trí thật, và chỉ ra chính xác giả thiết nào về hàm sai lệch g được dùng.
3. Nêu bốn chỗ mà triệt tiêu không hoàn toàn, và với mỗi chỗ nêu một biện pháp giảm nhẹ.
4. IBVS bền với sai số hiệu chuẩn. Chỉ ra chính xác chỗ mà K sai đi vào luật điều khiển, và vì sao chỗ ấy không ảnh hưởng tới điểm dừng.
5. Đi thẳng trục tốt cho điều khiển nhưng xấu cho đo góc. Nêu xung đột chính xác, và nêu giải pháp giải quyết được cả hai mà không hy sinh bên nào.
6. Stop-and-go có hai lợi ích. Nêu cả hai, nói cái nào lớn hơn và vì sao, và nêu một chi tiết thực hành hay bị quên khi cài đặt nó.
7. Côn dẫn hướng nói được dòng nào của ngân sách và không nói được dòng nào? Nêu một thay đổi thiết kế cơ khí giải quyết được cả dòng còn lại.

Nguồn

- [1] ISO, “ISO 9283:1998 — Manipulating Industrial Robots: Performance Criteria and Related Test Methods,” 1998, mức tin A; định nghĩa chuẩn của độ chính xác (accuracy) và độ lặp lại (repeatability) — nền của mục tách bias khỏi precision.
- [2] F. Chaumette and S. Hutchinson, “Visual Servo Control, Part I: Basic Approaches,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 4, pp. 82–90, 2006, mức tin A; tài liệu chuẩn cho IBVS và PBVS, ma trận tương tác.
- [3] D. E. Whitney, “Quasi-Static Assembly of Compliantly Supported Rigid Parts,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 104, no. 1, pp. 65–77, 1982, mức tin A; lý thuyết chamfer và remote center compliance — nguồn của luận điểm dung sai cơ khí thay cho độ chính xác thị giác.
- [4] B. Espiau, F. Chaumette, and P. Rives, “A New Approach to Visual Servoing in Robotics,” *IEEE Transactions on Robotics and Automation*, vol. 8, no. 3, pp. 313–326, 1992, mức tin A; bài gốc của IBVS, và nguồn của luận điểm IBVS bền với sai số hiệu chuẩn.
- [5] E. Malis, F. Chaumette, and S. Boudet, “2-1/2-D Visual Servoing,” *IEEE Transactions on Robotics and Automation*, vol. 15, no. 2, pp. 238–250, 1999, mức tin A.
- [6] F. Chaumette and S. Hutchinson, “Visual Servo Control, Part II: Advanced Approaches,” *IEEE Robotics & Automation Magazine*, vol. 14, no. 1, pp. 109–118, 2007, mức tin A; hybrid 2.5D và các trường hợp hỏng.
- [7] R. W. Brockett, “Asymptotic Stability and Feedback Stabilization,” *Differential Geometric Control Theory*, pp. 181–191, 1983, mức tin A; định lý Brockett — không có luật phản hồi trơn dừng nào ổn định hoá hệ nonholonomic.
- [8] C. Samson, “Control of Chained Systems: Application to Path Following and Time-Varying Point-Stabilization of Mobile Robots,” *IEEE Transactions on Automatic Control*, vol. 40, no. 1, pp. 64–77, 1995, mức tin A.
- [9] É. Marchand, F. Spindler, and F. Chaumette, “ViSP for Visual Servoing: A Generic Software Platform with a Wide Class of Robot Control Skills,” *IEEE Robotics & Automation Magazine*, vol. 12, no. 4, pp. 40–52, 2005, mức tin A.